

Dokumentacija sistema preporuka

Hibridni model koji spaja content-based rangiranje, item-based collaborative filtering i dinamičko rješavanje cold-start problema.

STUDENT	Imad Ali Mahović
BROJ INDEKSA	220336
PREDMET	Razvoj softvera II
DATUM	30.07.2026.

Kako sistem radi

Preporuka sale nastaje spajanjem osobina benda, osobina sale i historije rezervacija.

01

Vektor sadržaja

Žanr, broj članova, kapacitet, cijena, oprema i akustika.

02

Historija benda

Matrica bend × sala iz izvršenih i plaćenih rezervacija.

03

Hibridni rang

Kombinovani score sortira kandidate i vraća najrelevantnije sale.

HIBRIDNA FORMULA

$$\text{score} = \alpha * \text{content} + (1 - \alpha) * \text{collaborative}$$

Content-based signal

Cosine Similarity poredi vektor benda sa svakim vektorom sale. Rezultat je normalizovan na raspon 0-1. Kandidati se prije rangiranja filtriraju po aktivnosti, terminu, žanru, minimalnom kapacitetu i opremi.

Collaborative signal

Item-based adjusted cosine similarity pronalazi sale sa sličnim obrascem rezervacija. Ocjene se centriraju po bendu, a score kandidata je ponderisan sličnošću sa salama koje je bend ranije koristio.

Cold-start i dinamički alpha

Novi bend nema dovoljno historije, zato alpha počinje na 0.90 i daje veći uticaj sadržajnom modelu. Sa rastom historije eksponencijalno se smanjuje prema 0.35, čime collaborative signal postaje značajniji.

Glavna logika preporuke

Cosine Similarity, dinamički alpha, hibridna formula i rangiranje kandidata.

```

RecommendationEngine - content, cold-start i rangiranje
src/OpenAmp.Application/Recommendations/RecommendationEngine.cs

14 public sealed class RecommendationEngine
15 {
16     private const double MinimumAlpha = 0.35;
17     private const double MaximumAlpha = 0.90;
18     private const double ColdStartDecay = 4.0;
19
20     public static double CosineSimilarity(
21         IReadOnlyList<double> left,
22         IReadOnlyList<double> right)
23     {
24         ArgumentNullException.ThrowIfNull(left);
25         ArgumentNullException.ThrowIfNull(right);
26         if (left.Count != right.Count)
27         {
28             throw new ArgumentException("Vektori moraju imati isti broj elemenata.");
29         }
30
31         var dotProduct = 0d;
32         var leftNorm = 0d;
33         var rightNorm = 0d;
34         for (var index = 0; index < left.Count; index++)
35         {
36             dotProduct += left[index] * right[index];
37             leftNorm += left[index] * left[index];
38             rightNorm += right[index] * right[index];
39         }
40
41         if (leftNorm <= double.Epsilon || rightNorm <= double.Epsilon)
42         {
43             return 0;
44         }
45
46         return Math.Clamp(dotProduct / Math.Sqrt(leftNorm * rightNorm), -1, 1);
47     }
48
49     // ...
50     public static double DynamicAlpha(int historyCount)
51     {
52         ArgumentOutOfRangeException.ThrowIfNegative(historyCount);
53
54         var alpha = MinimumAlpha
55             + (MaximumAlpha - MinimumAlpha) * Math.Exp(-historyCount / ColdStartDecay);
56         return Math.Clamp(alpha, MinimumAlpha, MaximumAlpha);
57     }
58
59     public static double HybridScore(
60         double contentScore,
61         double collaborativeScore,
62         double alpha)
63     {
64         var boundedAlpha = Math.Clamp(alpha, 0, 1);
65         var boundedContent = Math.Clamp(contentScore, 0, 1);
66         var boundedCollaborative = Math.Clamp(collaborativeScore, 0, 1);
67         return boundedAlpha * boundedContent
68             + (1 - boundedAlpha) * boundedCollaborative;
69     }
70
71     public static IReadOnlyList<RecommendationScore> Rank(
72         int bandId,
73         IReadOnlyList<double> bandFeatures,
74         IReadOnlyCollection<RecommendationCandidate> candidates,
75         IReadOnlyCollection<RecommendationInteraction> interactions,
76         int limit)
77     {
78         ArgumentNullException.ThrowIfNull(bandFeatures);
79         ArgumentNullException.ThrowIfNull(candidates);
80         ArgumentNullException.ThrowIfNull(interactions);
81         if (limit <= 0)
82         {
83             return [];
84         }
85
86         var matrix = BuildMatrix(interactions);
87         var historyCount = matrix.TryGetValue(bandId, out var bandHistory)
88             ? bandHistory.Count
89             : 0;
90         var alpha = DynamicAlpha(historyCount);
91
92         return candidates
93             .Select(candidate =>
94             {
95                 var content = Math.Clamp(
96                     CosineSimilarity(bandFeatures, candidate.Features),
97                     0,
98                     1);
99                 var collaborative = CollaborativeScore(
100                     bandId,
101                     candidate.SaleId,
102                     matrix);
103                 return new RecommendationScore(
104                     candidate.SaleId,
105                     content,
106                     collaborative,
107                     alpha,
108                     HybridScore(content, collaborative, alpha));
109             })
110             .OrderByDescending(x => x.Score)
111             .ThenByDescending(x => x.ContentScore)
112             .ThenBy(x => x.SaleId)
113     }

```

Putanja: src/OpenAmp.Application/Recommendations/RecommendationEngine.cs

Prikazani su centralni dijelovi klase; brojevi linija odgovaraju stanju repozitorija pri izradi dokumenta.

Item-based collaborative filtering

Adjusted cosine similarity nad matricom bend × sala i ponderisanje poznatim preferencijama.

```

RecommendationEngine - item-based collaborative filtering
src/OpenAmp.Application/Recommendations/RecommendationEngine.cs

148     private static double AdjustedCosineSimilarity(
149         int firstHallId,
150         int secondHallId,
151         Dictionary<int, Dictionary<int, double>> matrix)
152     {
153         var numerator = 0d;
154         var firstNorm = 0d;
155         var secondNorm = 0d;
156         var commonBands = 0;
157
158         foreach (var bandRatings in matrix.Values)
159         {
160             if (!bandRatings.TryGetValue(firstHallId, out var firstRating)
161                 || !bandRatings.TryGetValue(secondHallId, out var secondRating))
162             {
163                 continue;
164             }
165
166             var mean = bandRatings.Values.Average();
167             var firstCentered = firstRating - mean;
168             var secondCentered = secondRating - mean;
169             numerator += firstCentered * secondCentered;
170             firstNorm += firstCentered * firstCentered;
171             secondNorm += secondCentered * secondCentered;
172             commonBands++;
173         }
174
175         if (commonBands < 2
176             || firstNorm <= double.Epsilon
177             || secondNorm <= double.Epsilon)
178         {
179             return 0;
180         }
181
182         return Math.Clamp(
183             numerator / Math.Sqrt(firstNorm * secondNorm),
184             -1,
185             1);
186     }
187
188     private static double CollaborativeScore(
189         int bandId,
190         int candidateHallId,
191         Dictionary<int, Dictionary<int, double>> matrix)
192     {
193         if (!matrix.TryGetValue(bandId, out var history) || history.Count == 0)
194         {
195             return 0.5;
196         }
197
198         var weightedScore = 0d;
199         var similarityWeight = 0d;
200         foreach (var (visitedHallId, preference) in history)
201         {
202             if (visitedHallId == candidateHallId)
203             {
204                 continue;
205             }
206
207             var similarity = AdjustedCosineSimilarity(
208                 visitedHallId,
209                 candidateHallId,
210                 matrix);
211             if (similarity <= 0)
212             {
213                 continue;
214             }
215
216             weightedScore += similarity * preference;
217             similarityWeight += similarity;
218         }
219
220         return similarityWeight <= double.Epsilon
221             ? 0.5
222             : Math.Clamp(weightedScore / similarityWeight, 0, 1);
223     }

```

Putanja: src/OpenAmp.Application/Recommendations/RecommendationEngine.cs
 Neutralna vrijednost 0.5 koristi se kada historija ili pozitivne sličnosti nisu dovoljne.

Preporuke u pokrenutoj aplikaciji

Snimak je napravljen iz finalnog Android release APK-a spojenog na lokalni OpenAmp API.

Šta je prikazano

Sekcija PREPORUČENO prikazuje rangirane sale iz endpointa `/api/recommendations/bands/{bandId}/halls`.

KORISNIK

muzicar (uloga MUZICAR)

PRVI REZULTAT

Jazz Corner

OBRAZLOŽENJE

Profil benda + historija

API ADRESA

`http://10.0.2.2:5264`

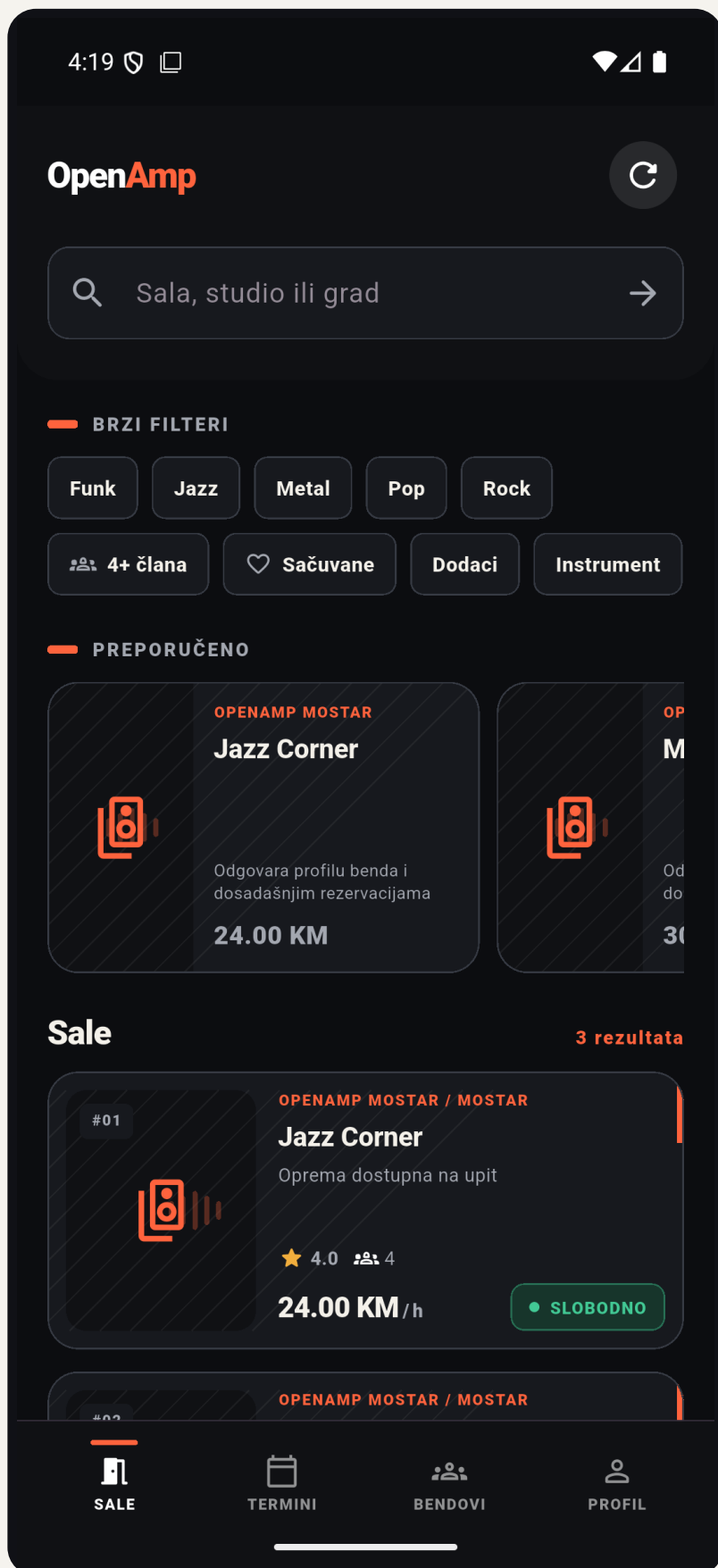
PUTANJE PRIKAZA

```
src/OpenAmp.Mobile/lib/screens/halls/hall_search_screen.dart
```

```
src/OpenAmp.Mobile/lib/state/app_state.dart
```

RELEASE APK

API 200 OK



Integracija i provjera

Preporuke nisu izolovan algoritam: servis ih veže za bazu, autorizaciju, filtere i Flutter UI.

API`src/OpenAmp.Api/Controllers/RecommendationsController.cs`

JWT endpoint za preporuke sale po bendu.

SERVIS`src/OpenAmp.Infrastructure/Recommendations/RecommendationService.cs`

Gradi vektore, matricu interakcija i objašnjenje rezultata.

ALGORITAM`src/OpenAmp.Application/Recommendations/RecommendationEngine.cs`

Cosine, item-based collaborative, alpha i hibridno rangiranje.

Završna provjera

44 / 44

.NET testovi

5 / 5

Flutter testovi

0

analyzer grešaka

2

release builda

Rezultat

Finalni Android APK je deinstaliran/ponovo instaliran u AVD-u, pokrenut i uspješno prijavljen na API. Windows release EXE je pokrenut iz vlastitog Release foldera i ostao responsive. Sistem preporuka je pokriven automatizovanim testovima i vidljiv u korisničkom interfejsu.